

✓ Project Python Foundations: FoodHub Data Analysis

Context

The number of restaurants in New York is increasing day by day. Lots of students and busy professionals rely on those restaurants due to their hectic lifestyles. Online food delivery service is a great option for them. It provides them with good food from their favorite restaurants. A food aggregator company FoodHub offers access to multiple restaurants through a single smartphone app.

The app allows the restaurants to receive a direct online order from a customer. The app assigns a delivery person from the company to pick up the order after it is confirmed by the restaurant. The delivery person then uses the map to reach the restaurant and waits for the food package. Once the food package is handed over to the delivery person, he/she confirms the pick-up in the app and travels to the customer's location to deliver the food. The delivery person confirms the drop-off in the app after delivering the food package to the customer. The customer can rate the order in the app. The food aggregator earns money by collecting a fixed margin of the delivery order from the restaurants.

Objective

The food aggregator company has stored the data of the different orders made by the registered customers in their online portal. They want to analyze the data to get a fair idea about the demand of different restaurants which will help them in enhancing their customer experience. Suppose you are hired as a Data Scientist in this company and the Data Science team has shared some of the key questions that need to be answered. Perform the data analysis to find answers to these questions that will help the company to improve the business.

Data Description

The data contains the different data related to a food order. The detailed data dictionary is given below.

Data Dictionary

- `order_id`: Unique ID of the order
- `customer_id`: ID of the customer who ordered the food

- `restaurant_name`: Name of the restaurant
- `cuisine_type`: Cuisine ordered by the customer
- `cost_of_the_order`: Cost of the order
- `day_of_the_week`: Indicates whether the order is placed on a weekday or weekend (The weekday is from Monday to Friday and the weekend is Saturday and Sunday)
- `rating`: Rating given by the customer out of 5
- `food_preparation_time`: Time (in minutes) taken by the restaurant to prepare the food. This is calculated by taking the difference between the timestamps of the restaurant's order confirmation and the delivery person's pick-up confirmation.
- `delivery_time`: Time (in minutes) taken by the delivery person to deliver the food package. This is calculated by taking the difference between the timestamps of the delivery person's pick-up confirmation and drop-off information

Please read the instructions carefully before starting the project.

This is a commented Jupyter IPython Notebook file in which all the instructions and tasks to be performed are mentioned. Read along carefully to complete the project.

- Blanks '***are provided in the notebook that needs to be filled with an appropriate code to get the correct result. Please replace the blank with the right code snippet. With every ' blank, there is a comment that briefly describes what needs to be filled in the blank space.***
- Identify the task to be performed correctly, and only then proceed to write the required code.
- Fill the code wherever asked by the commented lines like "# write your code here" or "# complete the code". Running incomplete code may throw an error.
- Please run the codes in a sequential manner from the beginning to avoid any unnecessary errors.
- You can the results/observations derived from the analysis here and use them to create your final presentation.

✓ Let us start by importing the required libraries

```
# Installing the libraries with the specified version.
!pip install numpy==1.25.2 pandas==1.5.3 matplotlib==3.7.1 seaborn==0.13
```

```

10.8/10.8 MB 67.1 MB/s eta
Installing build dependencies ... done
error: subprocess-exited-with-error
```

```

× Getting requirements to build wheel did not run successfully.
  exit code: 1
  ↳ See above for output.

note: This error originates from a subprocess, and is likely not a problem
Getting requirements to build wheel ... error
error: subprocess-exited-with-error

× Getting requirements to build wheel did not run successfully.
  exit code: 1
  ↳ See above for output.

note: This error originates from a subprocess, and is likely not a problem

```

Note:

- After running the above cell, kindly restart the runtime (for Google Colab) or notebook kernel (for Jupyter Notebook), and run all cells sequentially from the next cell.
- On executing the above line of code, you might see a warning regarding package dependencies. This error message can be ignored as the above code ensures that all necessary libraries and their dependencies are maintained to successfully execute the code in *this notebook*.

```

# Import libraries for data manipulation
import numpy as np
import pandas as pd

# Import libraries for data visualization
import matplotlib.pyplot as plt
import seaborn as sns

```

✓ Understanding the structure of the data

```

# uncomment and run the below code snippets if the dataset is present in
# from google.colab import drive
# drive.mount('/content/drive')

```

```

# Read the data
df = pd.read_csv('foodhub_order.csv') ## Fill the blank to read the data

```

```

# Returns the first 5 rows
df.head()

```

	order_id	customer_id	restaurant_name	cuisine_type	cost_of_the_order
0	1477147	337525	Hangawi	Korean	30.75
1	1477685	358141	Blue Ribbon Sushi Izakaya	Japanese	12.08
2	1477070	66393	Cafe Habana	Mexican	12.23
3	1477334	106968	Blue Ribbon Fried	American	29.20

Question 1: How many rows and columns are present in the data? [0.5 mark]

```
# Check the shape of the dataset
df.shape ## Fill in the blank
```

(1898, 9)

Question 2: What are the datatypes of the different columns in the dataset? [0.5 mark]

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1898 entries, 0 to 1897
Data columns (total 9 columns):
#   Column                      Non-Null Count  Dtype
---  -
0   order_id                    1898 non-null   int64
1   customer_id                 1898 non-null   int64
2   restaurant_name             1898 non-null   object
3   cuisine_type                1898 non-null   object
4   cost_of_the_order           1898 non-null   float64
5   day_of_the_week             1898 non-null   object
6   rating                      1898 non-null   object
7   food_preparation_time       1898 non-null   int64
8   delivery_time               1898 non-null   int64
dtypes: float64(1), int64(4), object(4)
memory usage: 133.6+ KB
```

✓ **Question 3:** Are there any missing values in the data? If yes, treat them using an appropriate method. [1 Mark]

```
# Checking for missing values in the data
df.isnull().sum() #Write the appropriate function to print the sum of n
```

	0
order_id	0
customer_id	0
restaurant_name	0
cuisine_type	0
cost_of_the_order	0
day_of_the_week	0
rating	0
food_preparation_time	0
delivery_time	0

dtype: int64

✓ **Question 4:** Check the statistical summary of the data. What is the minimum, average, and maximum time it takes for food to be prepared once an order is placed? [2 marks]

```
# Get the summary statistics of the numerical data
df.describe().round(2) ## Write the appropriate function to print the st
```

	order_id	customer_id	cost_of_the_order	food_preparation_time	de
count	1898.00	1898.00	1898.00	1898.00	
mean	1477495.50	171168.48	16.50	27.37	
std	548.05	113698.14	7.48	4.63	
min	1476547.00	1311.00	4.47	20.00	
25%	1477021.25	77787.75	12.08	23.00	
50%	1477495.50	128600.00	14.14	27.00	
75%	1477969.75	270525.00	22.30	31.00	
max	1478444.00	405334.00	35.41	35.00	

✓ **Question 5:** How many orders are not rated? [1 mark]

```
df['rating'].value_counts() ## Complete the code
```

	count
rating	
Not given	736
5	588
4	386
3	188

dtype: int64

Exploratory Data Analysis (EDA)

Univariate Analysis

Question 6: Explore all the variables and provide observations on their distributions. (Generally, histograms, boxplots, countplots, etc. are used for univariate exploration.) [9 marks]

✓ Order ID

```
# check unique order ID
df['order_id'].nunique()
```

1898

✓ Customer ID

```
# check unique customer ID
df['customer_id'].nunique() ## Complete the code to find out number of
```

1200

✓ Restaurant name

```
# check unique Restaurant Name
df['restaurant_name'].unique() ## Complete the code to find out number
#df['restaurant_name'].value_counts()
```

```
array(['Hangawi', 'Blue Ribbon Sushi Izakaya', 'Cafe Habana',
      'Blue Ribbon Fried Chicken', 'Dirty Bird to Go',
      'Tamarind TriBeCa', 'The Meatball Shop', 'Barbounia',
      'Anjappar Chettinad', 'Bukhara Grill',
      'Big Wong Restaurant \x8c_¼Ñ¼', 'Empanada Mama (closed)',
      'Pylos',
      "Lucky's Famous Burgers", 'Shake Shack', 'Sushi of Gari',
      'RedFarm Hudson', 'Blue Ribbon Sushi',
      'Five Guys Burgers and Fries', 'Tortaria', 'Cafe Mogador',
      'Otto Enoteca Pizzeria', 'Vezzo Thin Crust Pizza',
      'Sushi of Gari 46', 'The Kati Roll Company', 'Klong',
      '5 Napkin Burger', 'TA0', 'Parm', 'Sushi Samba',
      'Haru Gramercy Park', 'Chipotle Mexican Grill $1.99 Delivery',
      'RedFarm Broadway', 'Cafeteria', 'DuMont Burger',
      'Sarabeth's East', 'Hill Country Fried Chicken', 'Bistango',
      'Jack's Wife Freda', 'Mamoun's Falafel', 'Prosperity Dumpling',
      'Blue Ribbon Sushi Bar & Grill', 'Westville Hudson',
      'Blue Ribbon Brooklyn', 'Nobu Next Door', 'Osteria Morini',
      'Haandi', 'Benihana', 'Han Dynasty', 'Chote Nawab',
      'Mission Cantina', 'Xi'an Famous Foods', 'Rubirosa',
      "Joe's Shanghai \x8e_À\x8eüf¾÷", 'Bareburger', 'The Odeon',
      'Pongsri Thai', 'Yama Japanese Restaurant', 'Momoya',
      'Balthazar Boulangerie', 'Cafè China', 'Boqueria',
      'Song Thai Restaurant & Bar', 'Five Leaves',
      'Pinto Nouveau Thai Bistro', 'Amy Ruth's', 'Pepe Giallo',
      'indikitch', 'Yama 49', 'Piccolo Angolo', 'Pepe Rosso To Go',
      'L'Express', 'Amma', 'Delicatessen', 'S'MAC',
      'Vanessa's Dumplings', 'Bhatti Indian Grill', 'Taro Sushi',
      'Donburi-ya', 'Hatsuhana', 'Samurai Mama', 'Waverly Diner',
      'Tarallucci e Vino Restaurant', 'P.J. Clarke's',
      'Lantern Thai Kitchen', 'ilili Restaurant', 'The Smile',
      'Vanessa's Dumpling House', 'Bubby's ', 'Woorijip',
```

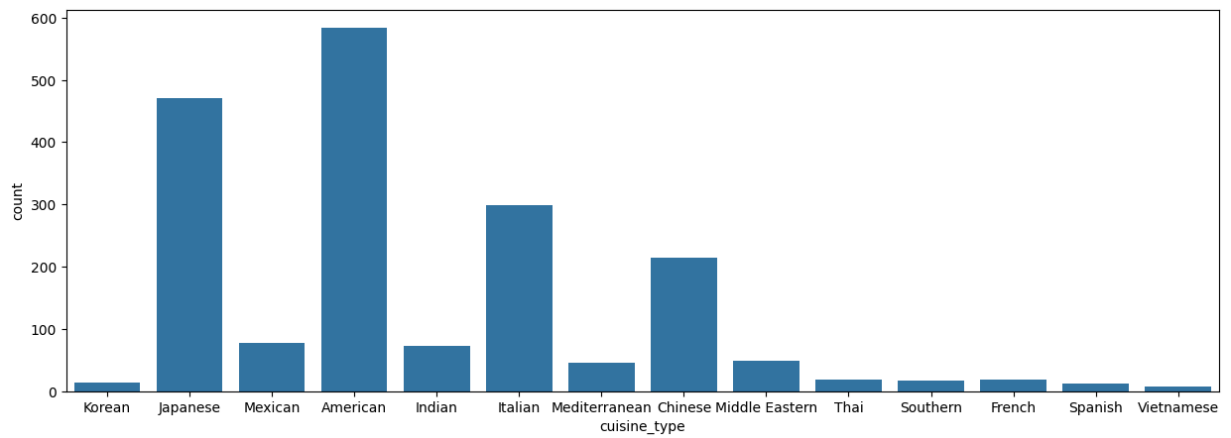
```
'Dirty Bird To Go (archived)', 'Haveli Indian Restaurant',
'Dos Caminos', 'da Umberto', 'Sushi of Gari Tribeca',
'Burger Joint', 'Room Service', 'Sarabeth's Restaurant',
'Xe May Sandwich Shop', 'Hibino', 'Mira Sushi', 'Melt Shop',
'J. G. Melon', 'Hummus Place', 'Saravanaa Bhavan',
'Friend of a Farmer', 'The Loop', 'Balade', 'Posto',
'Terakawa Ramen', 'Kambi Ramen House', 'Wo Hop Restaurant',
'Spice Thai', 'Dickson's Farmstand Meats',
'UVA Wine Bar & Restaurant', 'Serafina Fabulous Pizza',
'Gaia Italian Cafe', 'Chola Eclectic Indian Cuisine',
'Hot Kitchen', 'Junoon', 'Ravagh Persian Grill', 'Rohm Thai',
'Dig Inn Seasonal Market', 'Olea', 'Cho Dang Gol',
'El Parador Cafe', 'Socarrat Paella Bar',
'Don's Bogam BBQ & Wine Bar', 'Alidoro', 'Tony's Di Napoli',
'Cipriani Le Specialita', 'Sushi Choshi', 'Kanoyama', 'V-Nam
Cafe',
'Zero Otto Nove', 'Dos Caminos Soho', 'Go! Go! Curry!',
'La Follia', 'Izakaya Ten', '12 Chairs', 'Philippe Chow',
'The MasalaWala', 'brgr', 'Carmine's', 'Asuka Sushi', 'Aurora',
'Sarabeth's', 'Crema Restaurant', 'Big Daddy's',
'Moonstruck on Second', 'Cafe de La Esquina', 'Olive Garden',
'67 Burger', 'Tres Carnes', 'Schnipper's Quality Kitchen',
'Nha Trang One', 'Market Table', 'Galli Restaurant',
'Hampton Chutney Co.', 'Byblos Restaurant',
'Grand Sichuan International', 'Le Grainne Cafe', 'Il Bambino',
```

✓ Cuisine type

```
# Check unique cuisine type
df['cuisine_type'].unique() ## Complete the code to find out number of
#df['cuisine_type'].value_counts()
```

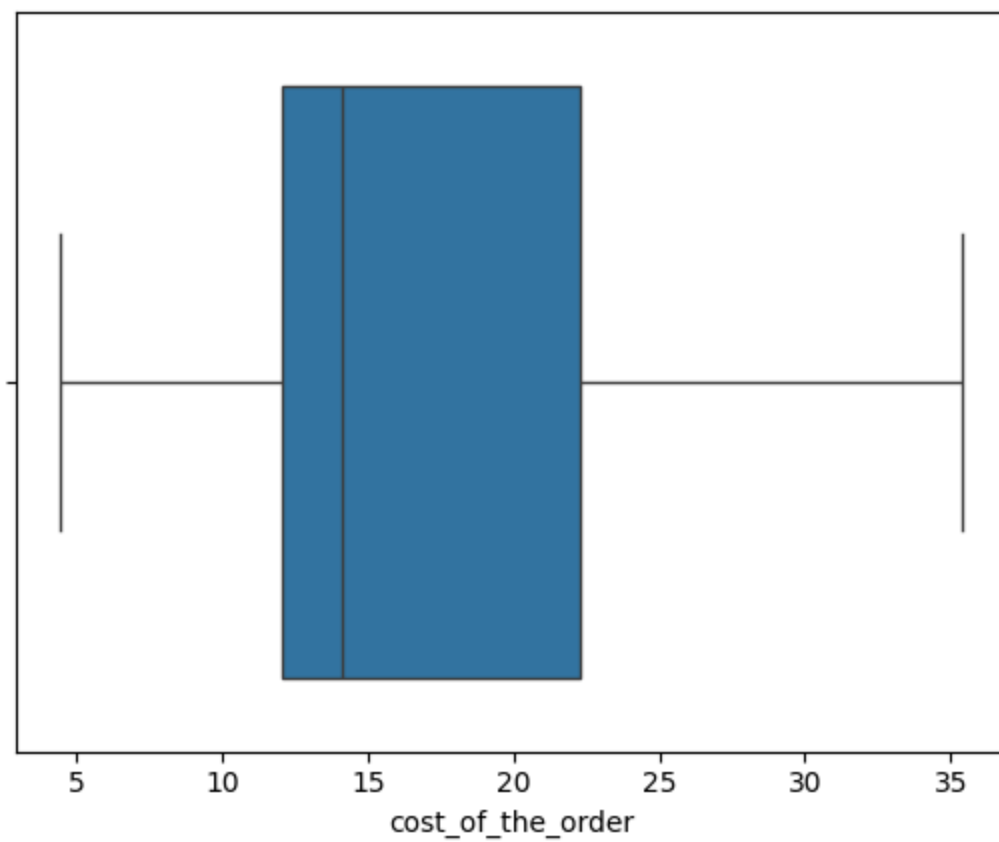
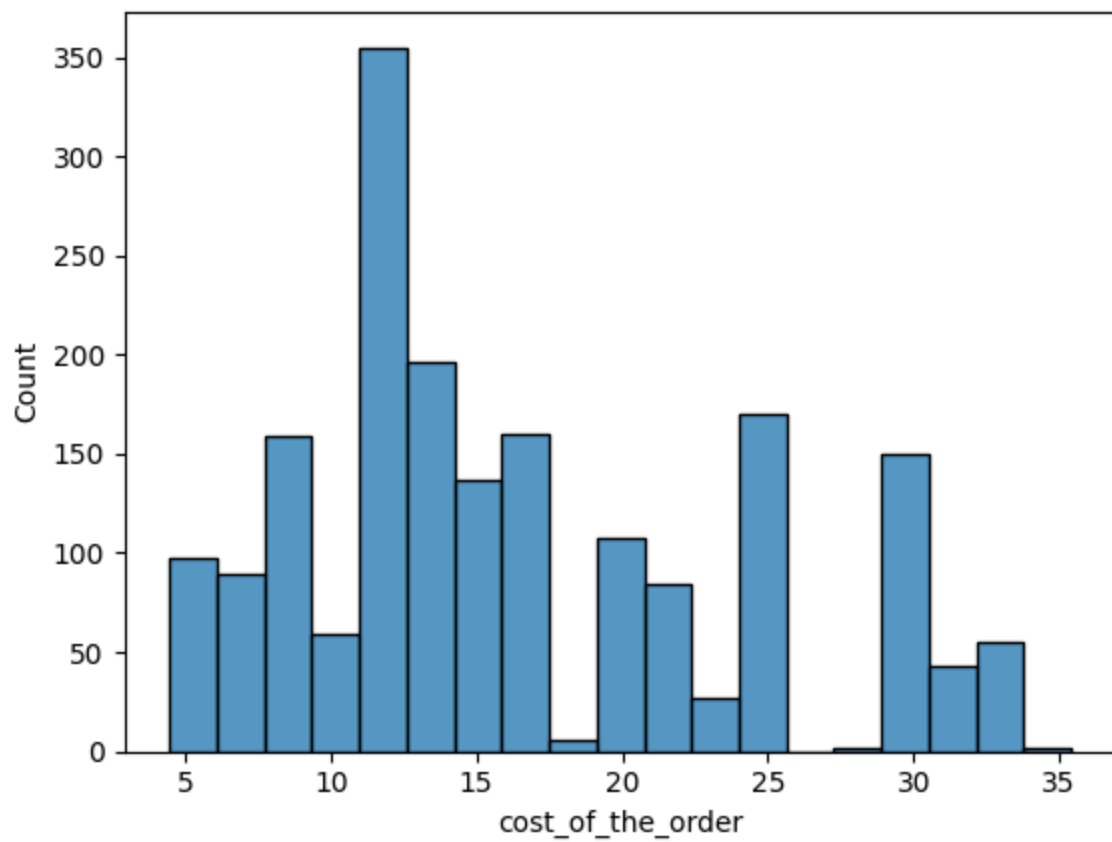
```
array(['Korean', 'Japanese', 'Mexican', 'American', 'Indian', 'Italian',
'Mediterranean', 'Chinese', 'Middle Eastern', 'Thai', 'Southern',
'French', 'Spanish', 'Vietnamese'], dtype=object)
```

```
plt.figure(figsize = (15,5))
sns.countplot(data = df, x = 'cuisine_type'); ## Create a countplot for
```



✓ Cost of the order

```
sns.histplot(data=df,x='cost_of_the_order') ## Histogram for the cost of  
plt.show()  
sns.boxplot(data=df,x='cost_of_the_order') ## Boxplot for the cost of or  
plt.show()
```

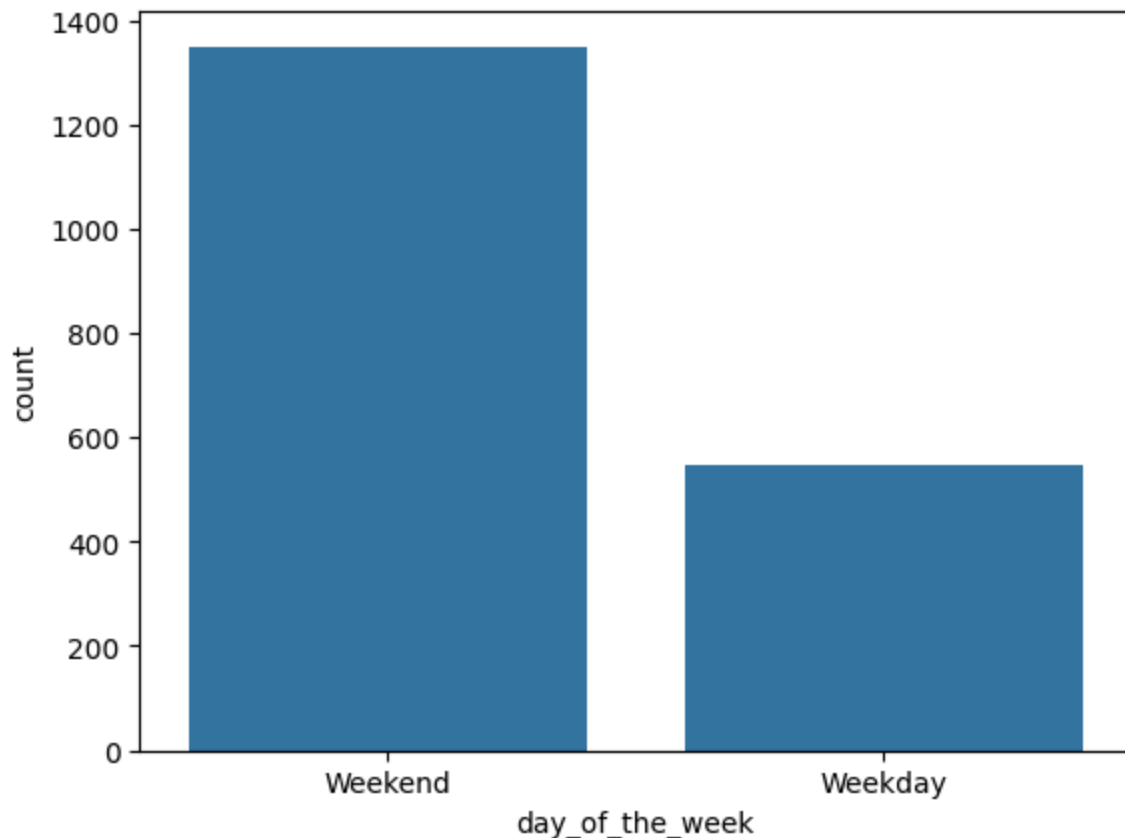


✓ Day of the week

```
# # Check the unique values
df['day_of_the_week'].unique() ## Complete the code to check unique valu

array(['Weekend', 'Weekday'], dtype=object)
```

```
sns.countplot(data = df, x = 'day_of_the_week'); ## Complete the code to
```

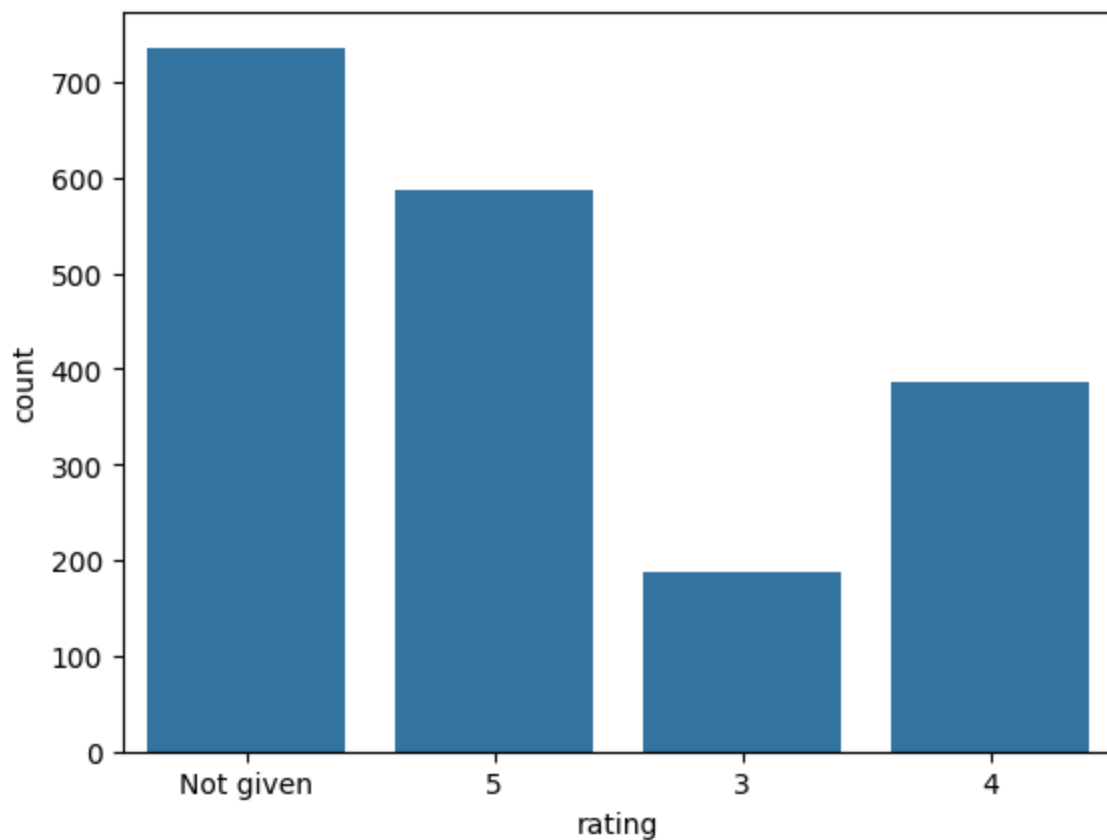


✓ Rating

```
# Check the unique values
df['rating'].unique () ## Complete the code to check unique values for t

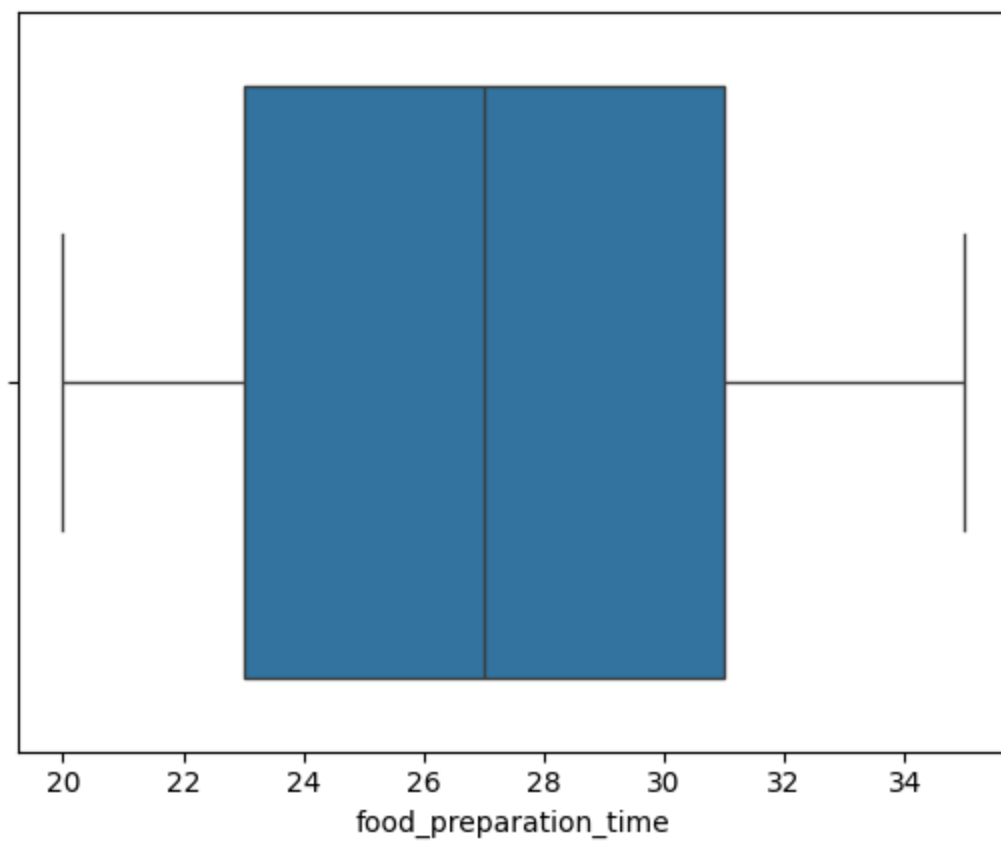
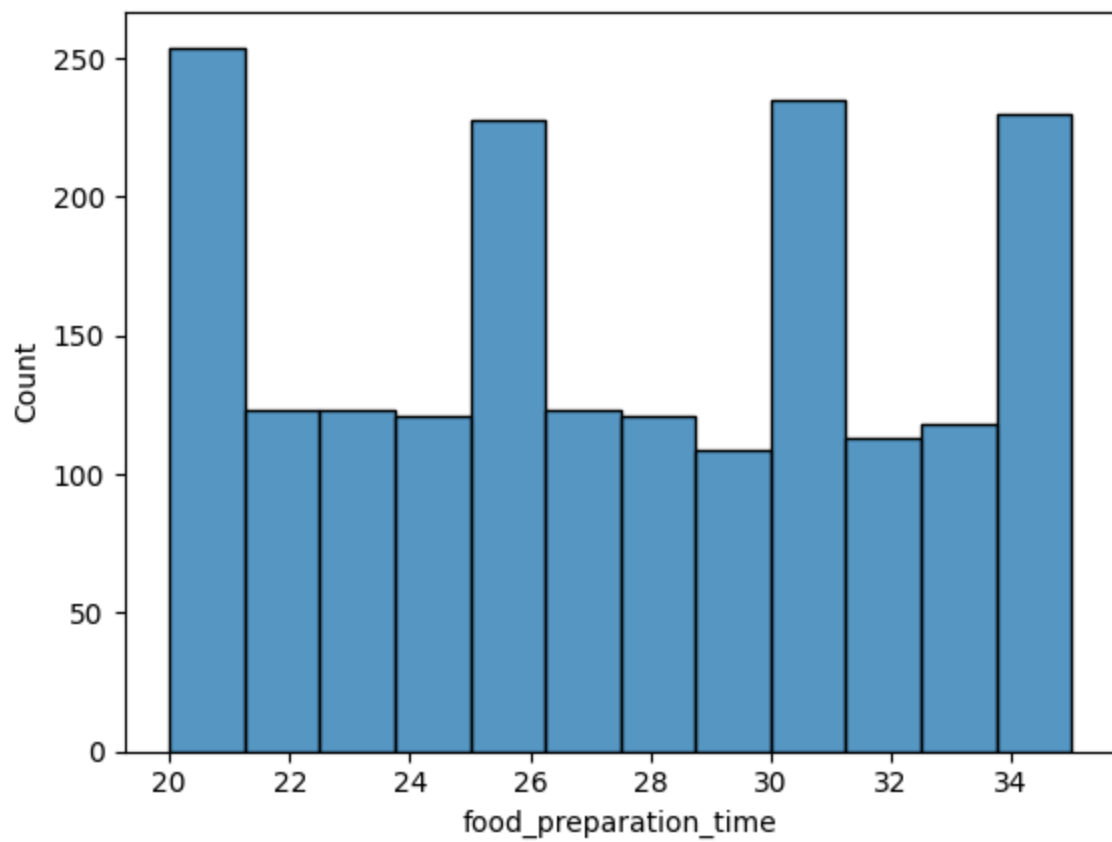
array(['Not given', '5', '3', '4'], dtype=object)
```

```
sns.countplot(data = df, x = 'rating'); ## Complete the code to plot bar
```



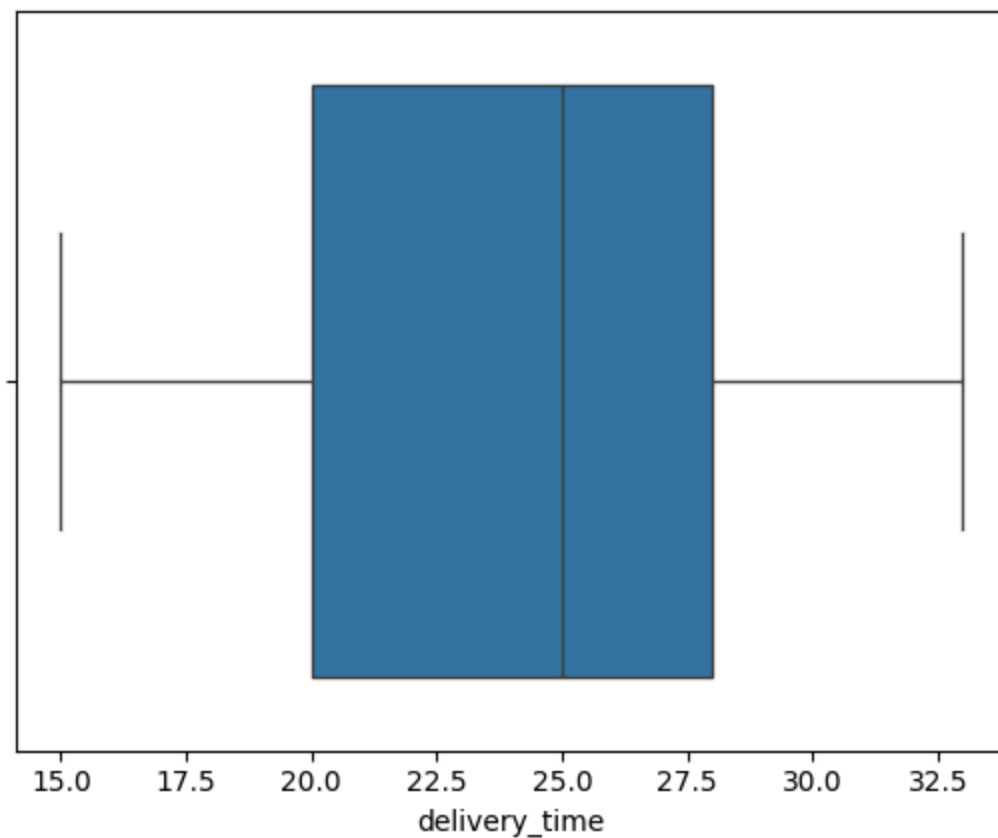
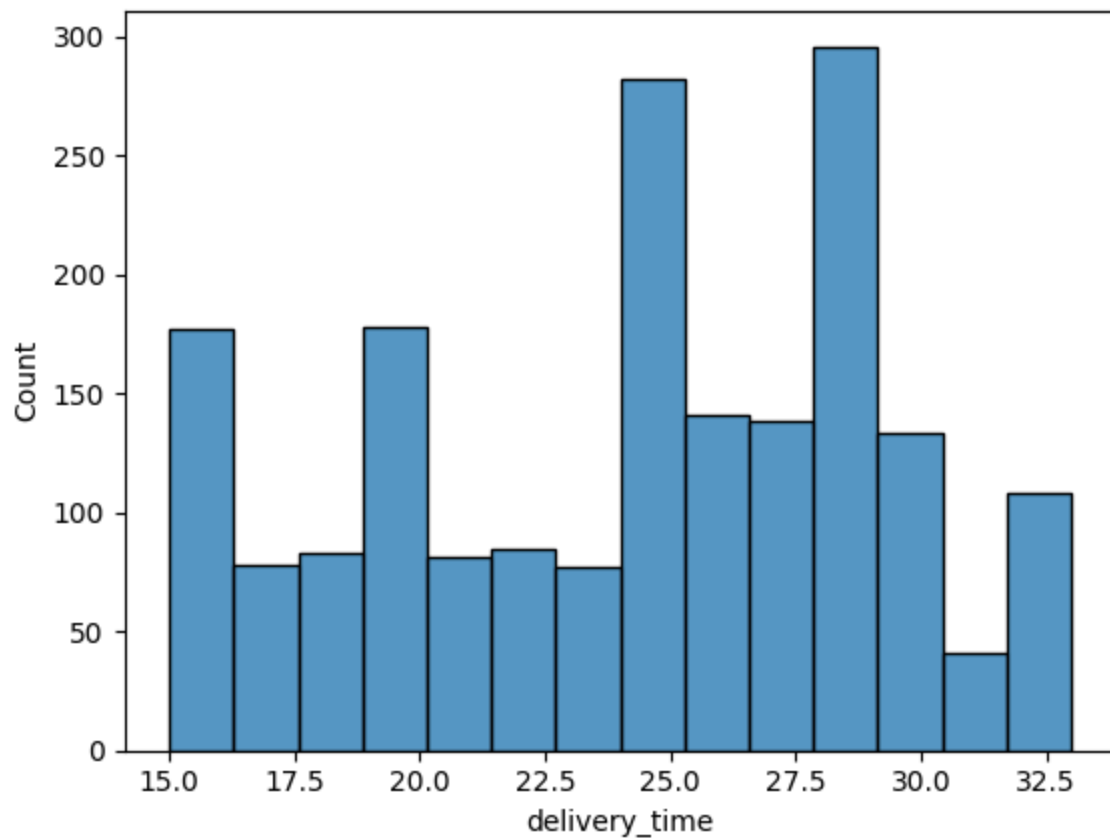
✓ Food Preparation time

```
sns.histplot(data=df,x='food_preparation_time') ## Complete the code to  
plt.show()  
sns.boxplot(data=df,x='food_preparation_time') ## Complete the code to p  
plt.show()
```



✓ Delivery time

```
sns.histplot(data=df,x='delivery_time') ## Complete the code to plot the  
plt.show()  
sns.boxplot(data=df,x='delivery_time') ## Complete the code to plot the  
plt.show()
```



- ✓ **Question 7:** Which are the top 5 restaurants in terms of the number of orders received? [1 mark]

```
# Get top 5 restaurants with highest number of orders
df['restaurant_name'].value_counts().head(5) ## Complete the code
```

	count
restaurant_name	
Shake Shack	219
The Meatball Shop	132
Blue Ribbon Sushi	119
Blue Ribbon Fried Chicken	96
Parm	68

dtype: int64

- ✓ **Question 8:** Which is the most popular cuisine on weekends? [1 mark]

```
# Get most popular cuisine on weekends
df_weekend = df[df['day_of_the_week'] == 'Weekend']
df_weekend['cuisine_type'].value_counts().head(15)
#df_weekend['cuisine_type'].unique()## Complete the code to check unique
```

	count
cuisine_type	
American	415
Japanese	335
Italian	207
Chinese	163
Mexican	53
Indian	49
Middle Eastern	32
Mediterranean	32
Thai	15
French	13
Korean	11
Southern	11
Spanish	11
Vietnamese	4

dtype: int64

✓ **Question 9:** What percentage of the orders cost more than 20 dollars?
[2 marks]

```
# Get orders that cost above 20 dollars
df_greater_than_20 = df[df['cost_of_the_order']>20] ## Write the appropriate code here

# Calculate the number of total orders where the cost is above 20 dollar
print('The number of total orders that cost above 20 dollars is:', df_greater_than_20.shape[0])

# Calculate percentage of such orders in the dataset
percentage = (df_greater_than_20.shape[0] / df.shape[0]) * 100

print("Percentage of orders above 20 dollars:", round(percentage, 2), '%')
```

```
The number of total orders that cost above 20 dollars is: 555
Percentage of orders above 20 dollars: 29.24 %
```

✓ **Question 10:** What is the mean order delivery time? [1 mark]

```
# Get the mean delivery time
mean_del_time = df['delivery_time'].mean() ## Write the appropriate fun

print('The mean delivery time for this dataset is', round(mean_del_time,

The mean delivery time for this dataset is 24.16 minutes
```

✓ **Question 11:** The company has decided to give 20% discount vouchers to the top 5 most frequent customers. Find the IDs of these customers and the number of orders they placed. [1 mark]

```
# Get the counts of each customer_id
df['customer_id'].value_counts().head(5) ## Write the appropriate colum
```

	count
customer_id	
52832	13
47440	10
83287	9
250494	8
259341	7

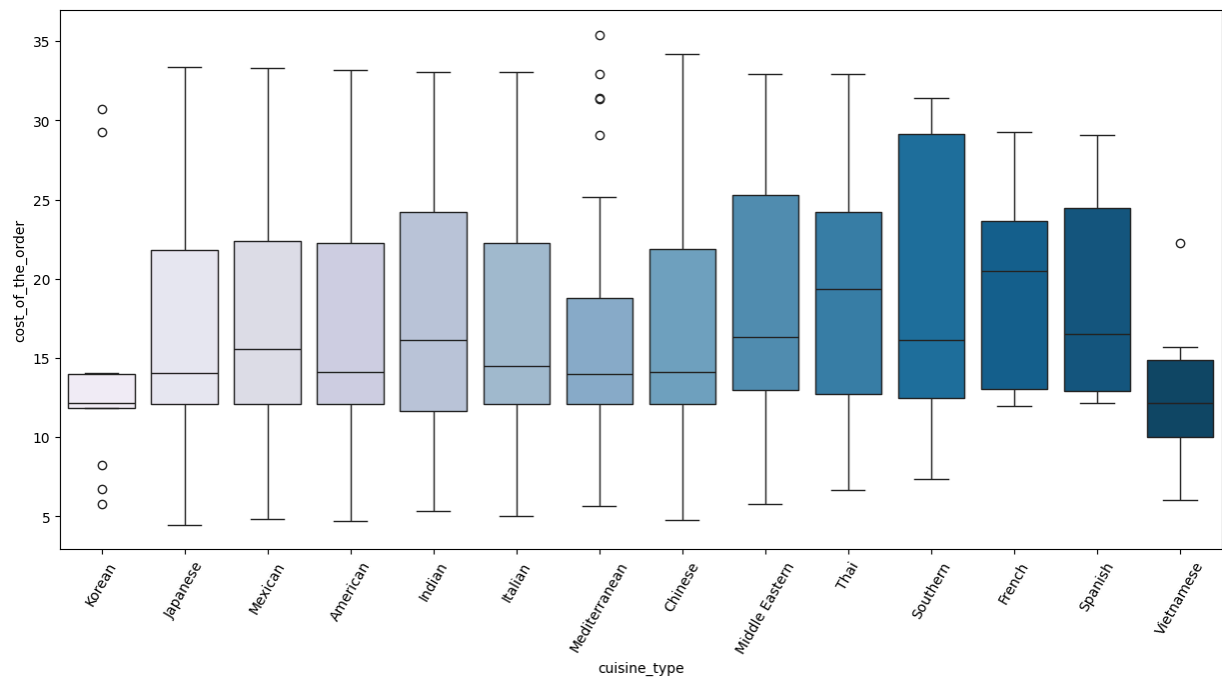
dtype: int64

Multivariate Analysis

✓ **Question 12:** Perform a multivariate analysis to explore relationships between the important variables in the dataset. (It is a good idea to explore relations between numerical variables as well as relations between numerical and categorical variables) [10 marks]

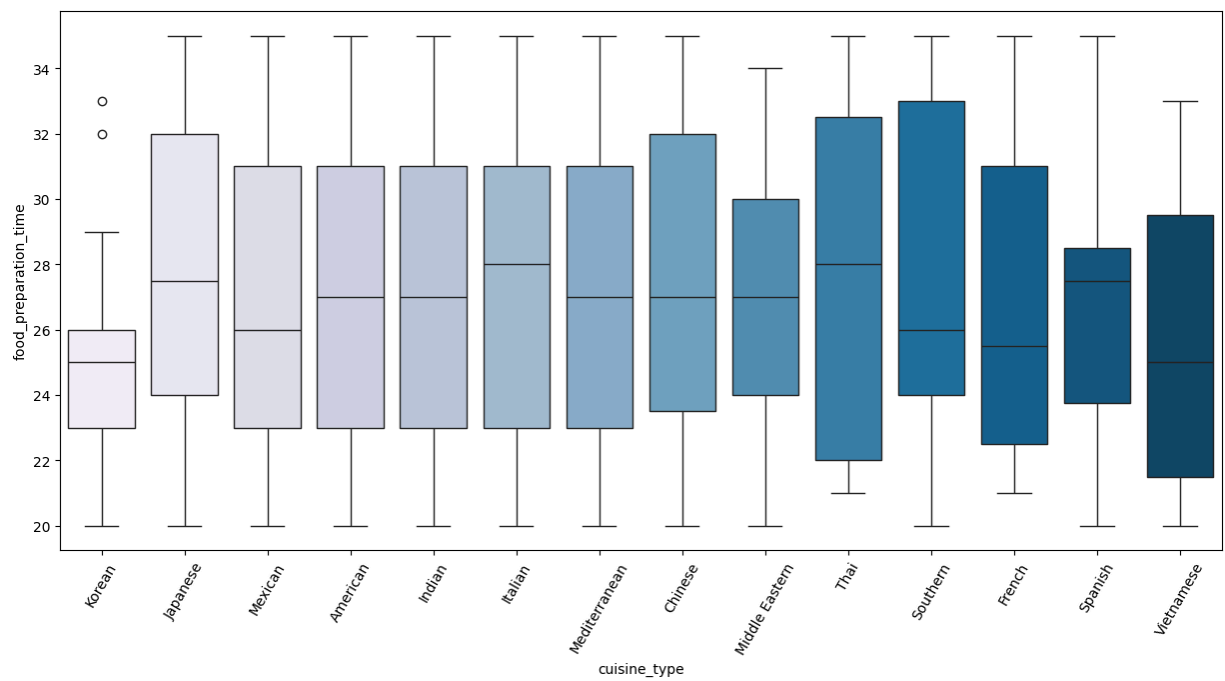
✓ Cuisine vs Cost of the order

```
# Relationship between cost of the order and cuisine type
plt.figure(figsize=(15,7))
sns.boxplot(x = "cuisine_type", y = "cost_of_the_order", data = df, palette
plt.xticks(rotation = 60)
plt.show()
```



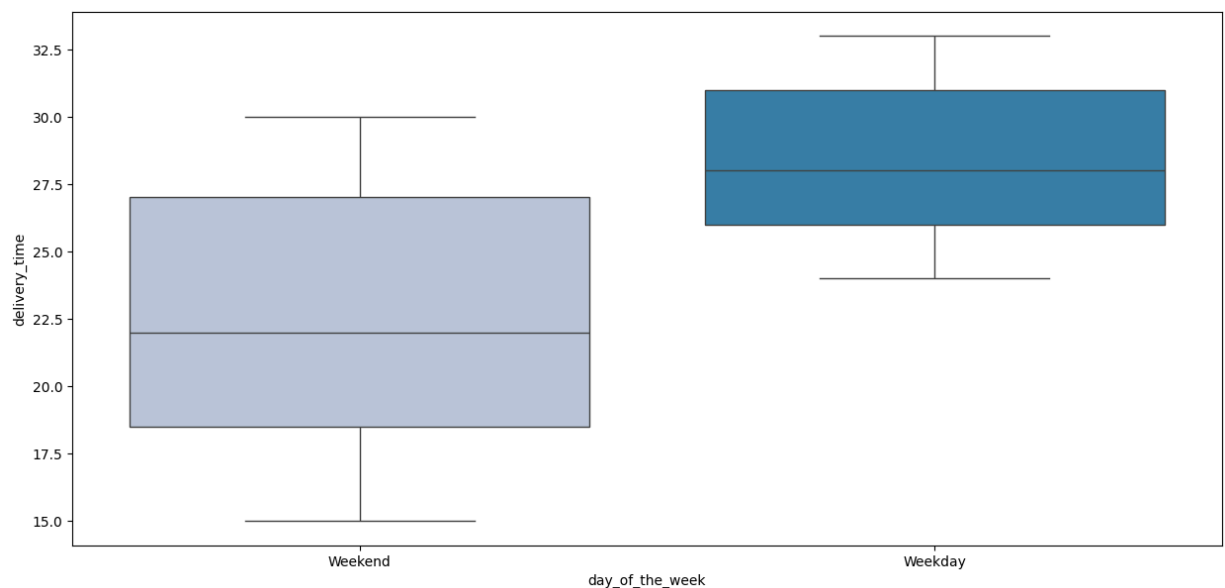
✓ Cuisine vs Food Preparation time

```
# Relationship between food preparation time and cuisine type
plt.figure(figsize=(15,7))
sns.boxplot(x='cuisine_type',y='food_preparation_time',data=df,palette='
plt.xticks(rotation = 60)
plt.show()
```



✓ Day of the Week vs Delivery time

```
# Relationship between day of the week and delivery time
plt.figure(figsize=(15,7))
sns.boxplot(x='day_of_the_week',y='delivery_time',data=df,palette='PuBu')
plt.show()
```



- Run the below code and write your observations on the revenue generated by the restaurants.

```
#df.groupby(['restaurant_name'])['cost_of_the_order'].sum().sort_values(
total_order_cost = df['cost_of_the_order'].sum()
print("Total order cost:", total_order_cost)
```

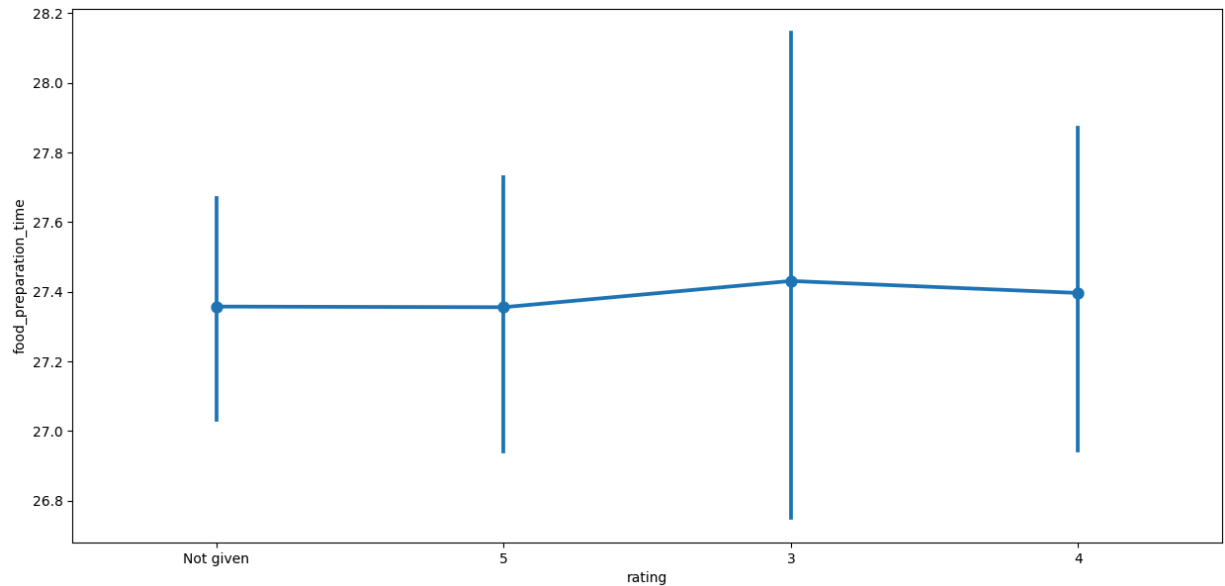
Total order cost: 31314.82

- Rating vs Delivery time

↳ 1 cell hidden

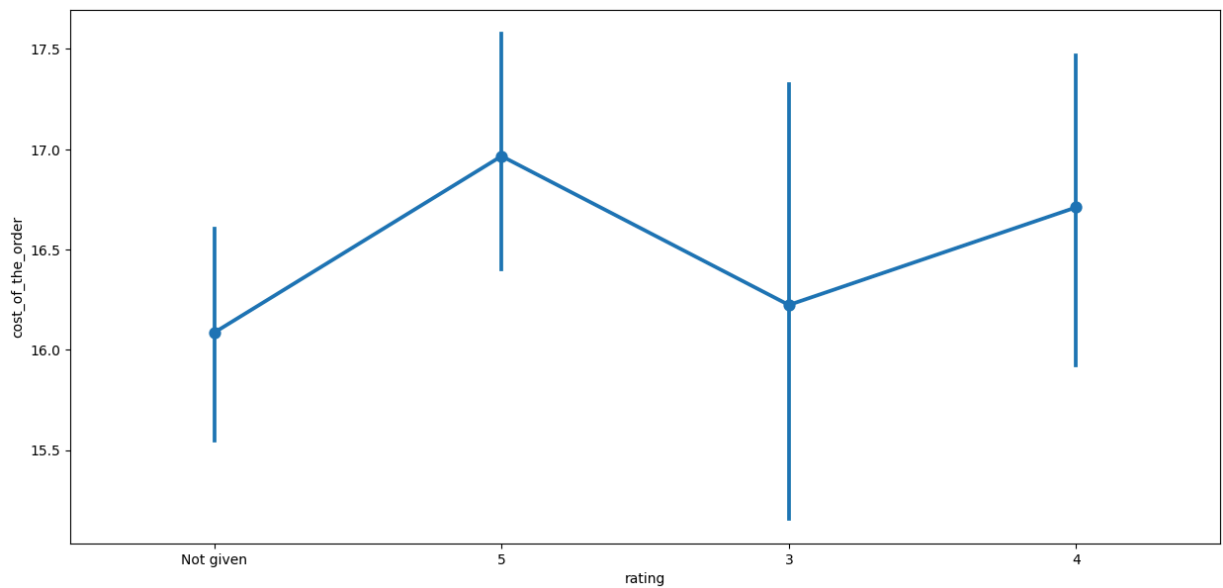
- Rating vs Food preparation time

```
# Relationship between rating and food preparation time
plt.figure(figsize=(15, 7))
sns.pointplot(x='rating', y='food_preparation_time', data=df) ## Complet
plt.show()
```



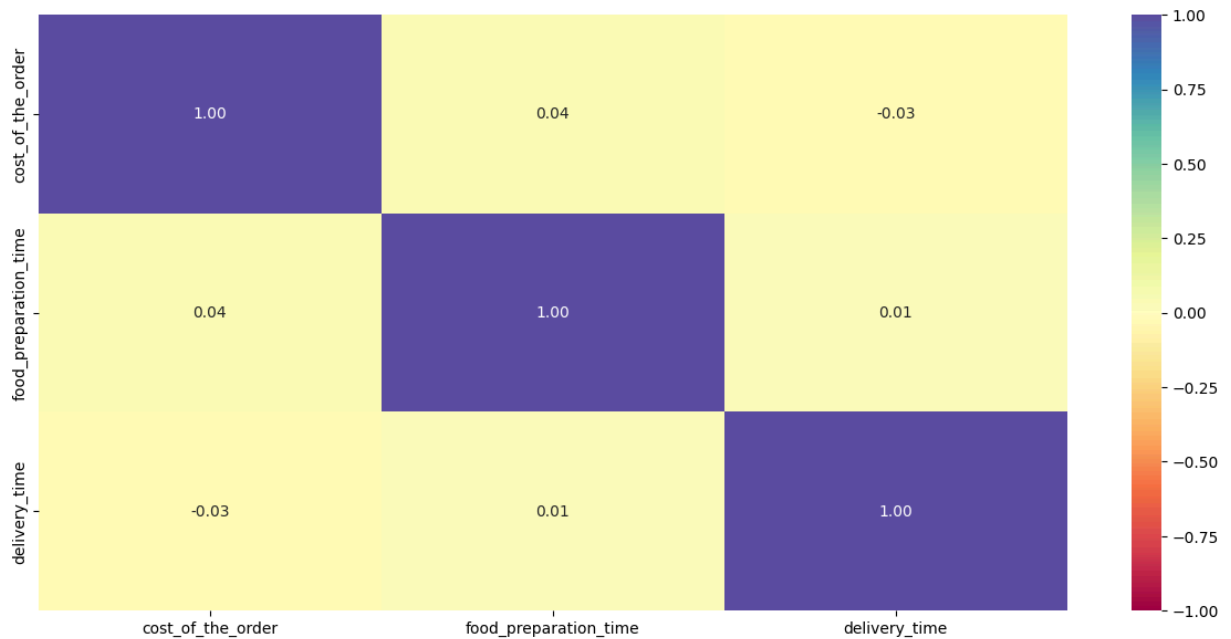
✓ Rating vs Cost of the order

```
# Relationship between rating and cost of the order
plt.figure(figsize=(15, 7))
sns.pointplot(x='rating', y='cost_of_the_order', data=df) ## Complete th
plt.show()
```



✓ Correlation among variables

```
# Plot the heatmap
col_list = ['cost_of_the_order', 'food_preparation_time', 'delivery_time']
plt.figure(figsize=(15, 7))
sns.heatmap(df[col_list].corr(), annot=True, vmin=-1, vmax=1, fmt=".2f",
plt.show())
```



Question 13: The company wants to provide a promotional offer in the advertisement of the restaurants. The condition to get the offer is that

- the restaurants must have a rating count of more than 50 and the average rating should be greater than 4. Find the restaurants fulfilling the criteria to get the promotional offer. [3 marks]

```
# Filter the rated restaurants
df Rated = df[df['rating'] != 'Not given'].copy()

# Convert rating column from object to integer
df Rated['rating'] = df Rated['rating'].astype('int')

# Create a dataframe that contains the restaurant names with their rating count
df Rating Count = df Rated.groupby(['restaurant_name'])['rating'].count()
df Rating Count.head()
```

	restaurant_name	rating
0	Shake Shack	133
1	The Meatball Shop	84
2	Blue Ribbon Sushi	73
3	Blue Ribbon Fried Chicken	64
4	RedFarm Broadwav	41

```
# Get the restaurant names that have rating count more than 50
rest_names = df_rating_count[df_rating_count['rating'] > 50]['restaurant_name']

# Filter to get the data of restaurants that have rating count more than 50
df_mean_4 = df_rated[df_rated['restaurant_name'].isin(rest_names)].copy()
```